Bienvenido

Este concurso es el Upsolving de la Segunda Ronda de la Copa Marco Reus - Farmeando Aura - Todas las Categorias.

Contenido

En este concurso podrás encontrar problemas 100 % lógicos, que te desafiaran.

Modalidad

El Contest durara 40 minutos y sera en formato Blitz.

Personal

Nombre	Rol
Juki	Admin
MarcoReus	Juez
MarcoReus	Staff
MarcoReus	Problem Setter

Cuadernos

Time Limit: 1 second **Memory Limit**: 256 MB

Type: Standard **Mode**: Total

Description

Sabes que 1 kg de pulpa de papel se puede utilizar para fabricar 1000 páginas y que cada cuaderno está compuesto por 100 páginas.

Supongamos que una fábrica de cuadernos recibe N kg de pulpa de papel. ¿Cuántos cuadernos se pueden fabricar con esa cantidad?

Input

- La primera línea contiene un entero T ($1 \leq T \leq 100$), que representa el número de casos de prueba.
- Cada caso de prueba contiene un único entero N ($1 \leq N \leq 100$), que representa la cantidad de pulpa de papel que tiene la fábrica, medida en kilogramos.

Output

Para cada caso de prueba, imprime en una sola línea la cantidad de cuadernos que se pueden fabricar utilizando N kg de pulpa de papel.

Input sample	Output sample
3 1 100 50	10 1000 500

Tipo de Dato

Time Limit: 1 second **Memory Limit**: 256 MB

Type: Standard **Mode**: Total

Description

Reus quiere almacenar algunos datos numéricos importantes en su computadora personal. Está utilizando un nuevo tipo de dato que solo puede almacenar valores desde 0 hasta N, ambos inclusive.

Si este tipo de dato recibe un valor mayor que N, el valor se ajusta de manera cíclica para mantenerse dentro del rango de 0 a N.

Por ejemplo:

- El valor $N + 1$ se almacenará como 0.
- El valor $N + 2$ se almacenará como 1.

Y así sucesivamente...

Dado X, el valor que Reus desea almacenar en este nuevo tipo de dato, determina cuál será el valor que realmente quedará almacenado en la memoria.

Input

- La primera línea contiene un entero T ($1 \leq T \leq 3000$), que representa el número de casos de prueba.
- Cada caso de prueba contiene una sola línea con dos enteros N ($1 \leq N \leq 50$) y X ($0 \leq X \leq 50$), separados por un espacio:
  - N: el valor máximo que puede almacenar el tipo de dato.
  - X: el valor que Reus desea almacenar.

Output

Para cada caso de prueba, imprime en una sola línea el valor que realmente se almacenará en la memoria.

Input sample	Output sample
5 15 0 15 10 11 12 27 37 50 49	0 10 0 9 49

Matriz Buena

Time Limit: 1 second **Memory Limit**: 256 MB

Type: Standard **Mode**: Total

Description

Reus se ha aficionado a las matrices cuadradas. Ahora está estudiando una matriz de tamaño $n \times n$, donde n es impar.

Reus considera que los siguientes elementos de la matriz son **buenos**:

- Los elementos de la diagonal principal.
- Los elementos de la diagonal secundaria.
- Los elementos de la fila "central" — la fila que tiene exactamente $\frac{n-1}{2}$ filas por encima y la misma cantidad de filas por debajo.
- Los elementos de la columna "central" — la columna que tiene exactamente $\frac{n-1}{2}$ columnas a su izquierda y la misma cantidad de columnas a su derecha.

La figura muestra una matriz de tamaño 5×5. Los elementos buenos están marcados en verde.

Ayuda a Reus a calcular la suma de todos los elementos buenos de la matriz dada.

Input

La primera línea contiene un único entero impar n.

Cada una de las siguientes n líneas contiene n enteros a_{ij} $(0 \leq a_{ij} \leq 100)$ separados por espacios — los elementos de la matriz dada.

- $1 \leq n \leq 101$

Output

Imprime un único entero — la suma de todos los elementos buenos de la matriz.

Input sample	Output sample
3 1 2 3 4 5 6 7 8 9	45
5 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	17

Llevame, llevame en tu Bicicleta

Time Limit: 1 second **Memory Limit**: 256 MB

Type: Standard **Mode**: Total

Description

La transmisión por cadena de la bicicleta de Reus consta de dos partes: **n** estrellas están sujetas al eje de los pedales y **m** estrellas están sujetas al eje de la rueda trasera. La cadena permite hacer girar la rueda trasera transmitiendo la rotación de los pedales.

Sabemos que la **i-ésima** estrella del eje de los pedales tiene $\mathbf{a_i}$ dientes $(0 < a_1 < a_2 < ... < a_n)$, y que la **j-ésima** estrella del eje de la rueda trasera tiene $\mathbf{b_j}$ dientes $(0 < b_1 < b_2 < ... < b_m)$.

Cualquier par **(i, j)** $(1 \le i \le n; 1 \le j \le m)$ se denomina **marcha** y determina los índices de las estrellas a las que está conectada actualmente la cadena.

La marcha **(i, j)** tiene una relación de transmisión igual a:

$\mathbf{b_j / a_i}$

Como a Reus le gustan los números enteros, quiere encontrar aquellas marchas **(i, j)** cuya relación sea un número entero.

Por otro lado, a Reus le gusta conducir rápido, por lo que, entre todas las marchas cuya relación sea un número entero, quiere elegir una marcha con la **máxima relación de transmisión**.

Ayúdalo a encontrar **cuántas marchas tienen la máxima relación de transmisión entre todas las marchas cuya relación es un número entero**.

En el problema, la fracción $\mathbf{b_j / a_i}$ representa una división en números reales, es decir, **no se realiza ningún redondeo**.

Input

La primera línea contiene un entero **n**$(1 \le n \le 50)$, que representa el número de estrellas en el eje de los pedales.

La segunda línea contiene **n** enteros:$\mathbf{a_1, a_2, ..., a_n}(1 \le a_i \le 10^4)$, dados en orden estrictamente creciente.

La tercera línea contiene un entero **m**$(1 \le m \le 50)$, que representa el número de estrellas en el eje de la rueda trasera.

La cuarta línea contiene **m** enteros:$\mathbf{b_1, b_2, ..., b_m}(1 \le b_j \le 10^4)$, dados en orden estrictamente creciente.

Se garantiza que existe al menos una marcha **(i, j)** cuya relación de transmisión es un número entero.

Los números de cada línea están separados por espacios.

Output

Imprime el número de marchas cuya relación de transmisión es **máxima entre todas las marchas con relación entera**.

Input sample Output sample

```
2
4 5
3
12 13 15
```

```
2
```

```
4
1 2 3 4
5
10 11 12 13 14
```

```
1
```

Puntos en el Plano

Time Limit: 1 second **Memory Limit**: 256 MB

Type: Standard **Mode**: Total

Description

Dados N puntos de la forma (x_i, y_i) en un plano bidimensional (2D).

Desde cada punto se trazan 2 líneas: una horizontal y otra vertical. Algunas de estas líneas pueden superponerse entre sí, por lo que debes determinar cuántas líneas **distintas** se pueden observar en el plano.

Nota:

- Dos líneas horizontales son distintas si pasan por diferentes coordenadas y.
- Dos líneas verticales son distintas si pasan por diferentes coordenadas x.

Input

- La primera línea contiene un entero T ($1 \leq T \leq 1000$), que representa el número de casos de prueba.
- Cada caso de prueba comienza con un entero N ($1 \leq N \leq 10^5$), que representa la cantidad de puntos.
- Las siguientes N líneas contienen dos enteros x_i y y_i ($0 \leq x_i, y_i \leq 10^9$), separados por un espacio, que representan las coordenadas del i-ésimo punto.
- La suma de N entre todos los casos de prueba no supera 10^5.

Output

Para cada caso de prueba, imprime en una sola línea la cantidad de líneas **distintas** que se pueden observar en el plano.

Input sample

```
3
4
1 1
1 0
0 1
0 0
5
0 0
0 1
0 2
0 3
0 4
```

Output sample

```
4
6
2
```

```
1
10 10
```

Juego Rectangular

Time Limit: 1 second **Memory Limit**: 256 MB

Type: Standard **Mode**: Total

Description

Reus decidió tomarse un día libre. Sin embargo, no hacer nada durante todo el día resultó ser demasiado aburrido, así que decidió jugar con unas piedras.

Inicialmente, Reus tiene n piedras. Las organiza en a filas iguales, donde cada fila contiene b piedras y $a > 1$.

Reus debe utilizar todas las piedras que tiene, es decir:

$n = a \cdot b$

Por ejemplo, 10 piedras pueden organizarse en 2 filas, con 5 piedras en cada fila.

Una vez que Reus ha organizado las piedras, toma cualquiera de las filas obtenidas, es decir, se queda con b piedras, y descarta todas las demás.

Luego vuelve a organizar **todas** las piedras que le quedan, posiblemente eligiendo otros valores de a y b, toma nuevamente una fila y descarta el resto.

El juego continúa de esta manera hasta que, en algún momento, Reus termina con exactamente una piedra.

El proceso del juego puede representarse mediante una secuencia finita de enteros $c_1, \ldots, c_k$, donde:

- $c_1 = n$.
- c_{i+1} es la cantidad de piedras con las que Reus termina después del i-ésimo movimiento, es decir, la cantidad de piedras que contiene una fila después de organizar las c_i piedras, para $1 \leq i < k$.
- Se cumple que $c_i > c_{i+1}$.
- $c_k = 1$.

El resultado del juego es la suma de todos los valores c_i.

Dado n, encuentra el máximo resultado posible del juego.

Input

La única línea de entrada contiene un entero n — la cantidad inicial de piedras que tiene Reus.

- $2 \leq n \leq 10^9$

Output

Imprime un único número — el máximo resultado posible del juego.

Input sample	Output sample
10	16

8

15

!Afortunado

Time Limit: 1 second **Memory Limit**: 256 MB

Type: Standard **Mode**: Total

Description

Cada uno de ustedes probablemente tiene su propia experiencia viajando en transporte público y comprando boletos. Después de que una persona compra un boleto (que tradicionalmente tiene un **número par de dígitos**), normalmente comprueba si el boleto es afortunado.

Recordemos que un boleto es $afortunado$ si la suma de los dígitos de su primera mitad es igual a la suma de los dígitos de su segunda mitad.

Pero, por supuesto, no todos los boletos pueden ser afortunados. ¡Ni mucho menos! Además, a veces basta con mirar un boleto una sola vez para saber inmediatamente que no es afortunado.

Por eso, consideremos el siguiente **criterio de infortunio**, que permite determinar con certeza que un boleto no es afortunado.

Diremos que un boleto es **definitivamente desafortunado** si cada dígito de la primera mitad puede corresponderse con algún dígito de la segunda mitad, de manera que:

- cada dígito de la primera mitad sea **estrictamente menor** que el dígito correspondiente de la segunda mitad;

o

- cada dígito de la primera mitad sea **estrictamente mayor** que el dígito correspondiente de la segunda mitad.

Cada dígito debe utilizarse **exactamente una vez** en las comparaciones.

En otras palabras, debe existir una **correspondencia biyectiva** entre los dígitos de la primera y segunda mitad del boleto, tal que se cumpla una de las siguientes condiciones:

- todos los dígitos de la primera mitad son estrictamente menores que sus correspondientes de la segunda mitad;
- todos los dígitos de la primera mitad son estrictamente mayores que sus correspondientes de la segunda mitad.

Por ejemplo, el boleto 2421 cumple el criterio de infortunio y, por lo tanto, no se considera afortunado. La correspondencia buscada es:

$2 > 1$ y $4 > 2$.

El boleto 0135 también cumple el criterio, ya que podemos establecer la correspondencia:

$0 < 3$ y $1 < 5$.

En cambio, el boleto 3754 no cumple este criterio.

Reus tiene un boleto en sus manos que contiene **2·n dígitos**. Tu tarea es determinar si cumple el criterio de infortunio.

Input

La primera línea contiene un entero n $(1 \leq n \leq 100)$.

La segunda línea contiene una cadena formada por $2 * n$ dígitos, que representa el boleto.

Output

En la primera línea imprime: YES si el boleto cumple el criterio de infortunio.

De lo contrario, imprime: NO

Input sample	Output sample
2 2421	YES
2 0135	YES
2 3754	NO

La Carta

Time Limit: 1 second **Memory Limit**: 256 MB

Type: Standard **Mode**: Total

Description

Reus acaba de terminar de escribir un mensaje para su enamorada cuando se dio cuenta de que el mensaje no se veía lo suficientemente elegante. Reus estaba nervioso mientras lo escribía, por lo que algunas letras quedaron en minúsculas y otras en mayúsculas.

Reus considera que un mensaje es **elegante** si todas las letras mayúsculas aparecen antes que cualquier letra minúscula. En otras palabras, un mensaje elegante es una cadena que comienza con cero o más letras mayúsculas, seguidas de cero o más letras minúsculas.

Para hacer que el mensaje sea elegante, Reus puede cambiar una letra de mayúscula a minúscula o viceversa, manteniéndola en la misma posición.

Cada cambio de una letra cuenta como una acción. Reus no puede realizar ningún otro tipo de operación.

Ayuda a Reus a encontrar la cantidad mínima de acciones necesarias para convertir su mensaje en uno elegante.

Input

La única línea de entrada contiene una cadena no vacía formada por letras mayúsculas y minúsculas del alfabeto inglés.

La longitud de la cadena no supera 10^5.

Output

Imprime un único entero — la cantidad mínima de acciones necesarias para convertir el mensaje de Reus en uno elegante.

Input sample	Output sample
COpamaRCOfAR	5
COPAMARCOREUSFARMEANDOAURA	0
holamundo	0